TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

Programming in Java – Assignment 1 (CO1) | Day-4 Design Challenge | 8 Marks | K5 – Evaluate

1. Introduction

The original Campus Smart-Parking System supports two-wheelers, cars and electric cars. The university now introduces e-bikes with a flat fee, delivery vans charged at twice the car rate during 9 AM–5 PM, visitor one-day passes, and a five-floor facility with unequal slot capacities. The original design must be evaluated and adapted to satisfy these requirements.

A. Impact Analysis – 2 Marks

The new requirements affect vehicle categories, pricing, pass validity, ticket timestamps and facility configuration. The Vehicle hierarchy must support e-bikes and delivery vans. Pricing logic must support a flat fee and time-dependent hourly rates. Visitor pass validation must be added. ParkingFacility must support five floors with different capacities, which is naturally represented by a jagged array.

Component	Impact / required change
Vehicle hierarchy	Add Ebike and DeliveryVan implementations.
Pricing	Separate pricing strategies for hourly, flat and time-dependent rules.
Parking pass	Add a ParkingPass abstraction and OneDayPass implementation.
ParkingTicket	Store entry time, floor, slot and unique ticket number.
ParkingFacility	Configure five unequal floors and manage active tickets.
Active-ticket storage	Use HashMap keyed by registration number for average O(1) lookup.

A jagged array is appropriate because each floor can have a different number of slots. A rectangular array would reserve equal capacity on every floor and could waste space.

B. Evaluation of the Original Design – 2 Marks

The original design is partially suitable because it already uses an abstract Vehicle class, subclasses and polymorphism. However, placing all new pricing conditions inside vehicle subclasses or a large conditional statement would increase coupling. The original design should be modified, not completely discarded. New vehicle categories can be added as subclasses, while pricing behaviour should be separated into independent strategy objects. The slot-search logic can remain substantially unchanged; configuration and ticket storage need adaptation.

C. Adaptation or Redesign – 2 Marks

The selected approach is modification using a PricingRule interface and separate implementations: HourlyPricingRule, EbikeFlatPricingRule and DeliveryVanPricingRule. A ParkingPass interface and OneDayPass implementation represent visitor pass validity. Vehicle objects receive a pricing rule, so the facility does not need to know the detailed pricing formula. The facility uses a five-floor jagged array and a HashMap for active tickets.

Revised component	Responsibility
Vehicle	Common abstract vehicle data and pricing delegation.
Ebike / DeliveryVan	New vehicle categories using their respective pricing strategies.
PricingRule	Common fee-calculation contract.
OneDayPass	Checks whether a visitor pass covers the parking date.
ParkingFacility	Manages allocation, tickets, fee calculation and release.
ParkingTicket	Stores ticket number, registration, entry time and slot.

Pricing assumptions used in the demonstration: cars cost Rs. 30 per started hour; e-bikes cost a flat Rs. 25 per session; delivery vans cost Rs. 30 per started hourly block outside 9 AM–5 PM and Rs. 60 per block whose start time falls within 9 AM–5 PM. A valid one-day pass covers its specified calendar date. These are explicit implementation assumptions for demonstrating the changed requirements.

D. Evidence-Based Defence – 2 Marks

Evidence	Result
E-bike test	Flat fee of Rs. 25.
Delivery van, 8–10 AM	Rs. 30 + Rs. 60 = Rs. 90.
Visitor with valid one-day pass	Rs. 0.
Standard car, 2 hours	2 × Rs. 30 = Rs. 60.
Five-floor configuration	Unequal capacities: 2, 3, 4, 2 and 5 slots.
After exit	Active ticket count returns to zero.

The modification introduces additional classes and interfaces, but reduces the need to edit existing facility-management code when pricing rules change. A future vehicle type can be added by creating a vehicle class and assigning an appropriate PricingRule. The trade-off is greater initial class count and additional HashMap memory in exchange for extensibility and faster average ticket lookup.

4. Complexity Analysis

Operation	Array tickets	HashMap tickets
Find ticket	O(n)	O(1) average
Add ticket	O(1) if capacity exists	O(1) average
Remove ticket	O(n)	O(1) average
Storage	O(n)	O(n)
Find free slot	O(S)	O(S) with same scan

Here n is the number of active tickets and S is the total number of parking slots. The delivery-van pricing strategy takes O(h) time for h billable hourly blocks because it checks each block's start time. Standard hourly and flat-fee calculations take O(1) time.

5. Conclusion

The original design should be modified rather than completely redesigned. Pricing strategies, a pass abstraction, a jagged five-floor array and HashMap-based ticket management address the changed requirements while preserving the core object-oriented model. The tests demonstrate the new e-bike, delivery-van and one-day-pass behaviour, supporting the design decision.